A leaderboard that cannot measure: negative results and measurement limits in the AIMO Interpretability Challenge

Oleksandr Barskyi
Independent researcher
oleksandr.umka.barskyi@gmail.com

Abstract

We report a fully negative but, we argue, informative participation in the AIMO Interpretability Challenge. Across 28 submissions we never beat the best constant by an amount the test can distinguish from noise, and our central claim is that on private tests of 14 and 21 cases *no one can be shown to*, at the accuracy label-free signals reach here. A scorer with ROC AUC 0.65 is declared better than the best constant in 8.5% of runs on the 14-case Small Models Track and 15.6% on the 21-case Main Track; 150–200 cases are needed for 80% power. Four fifths of the variance of a leaderboard entry is which cases were drawn, not what the method did. On the organizers' own 80 olympiad problem–model pairs no robust pair has intermediate base accuracy and 16 of 25 robust pairs have base accuracy exactly zero: "robust" largely means "fails identically", which explains why the published reference solution scores below both constants and inverts the intuitive sign of a self-consistency proxy. Finally, the per-group exception handling of ingestion turns the scoring output into an oracle for predicates over the private test; we used it only to count the composition of a since-retired test, flagged it to the organizers on 2026-09-19, and describe it here in full. All code and raw scores are public.

1 Introduction

The AIMO Interpretability Challenge asks for a binary verdict per (problem, model) pair: does the model answer this olympiad problem *robustly*, or does the answer rest on a pattern that perturbation removes? Submissions are ranked by accuracy on a held-out test, separately on a Main Track (five models, including `openai/gpt-oss-120b`) and a Small Models Track (the four below 10B parameters). The write-up prizes are judged "regardless of ranking", with explicit interest in negative results; this report is written for them.

Our position shapes everything below: a single participant on a CPU-only server, with no GPU at any point, so every generation-based measurement had to happen *inside* an evaluation submission, with empty logs and a three-number return channel. We claim no place on either leaderboard: our best scores are 0.5714 (8/14) on the Small track, by a method we believe is indistinguishable from a coin, and 0.5238 (11/21) on the Main track, by the constant `always-false`. We submitted no white-box probe: we could not train one, and we say so rather than dress a behavioural method in interpretability language.

Contributions. (1) A power analysis of the challenge's own leaderboards and a decomposition of a score's variance into case draw and run (§ 3), checked against the live board. (2) A diagnosis of why the official reference solution scores below a constant (§ 4), which generalises to any method calibrated on `hendrycks MATH`-style data and evaluated at olympiad difficulty. (3) A measured, not assumed, sign for a label-free robustness proxy (§ 5). (4) The scoring channel as a predicate oracle, with a proposed mitigation (§ 6). (5) Eleven pre-registered predictions

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

with their outcomes, and an appendix of engineering failure modes invisible without local replication of the container (Appendices A, F). Code, every submitted bundle and every raw score: `https://github.com/oleksandr-barskyi/aimo-interp-report`.

Related work. § 3 instantiates the argument of Card et al. [1] on underpowered test sets and the variance accounting of Bouthillier et al. [2], specialised to a 14-case leaderboard where the case draw dominates. § 6 relates to leaderboard probing and adaptive overfitting [3–5] and adds a *coverage* channel to the known label channel. § 5 uses self-consistency [6] as a robustness signal with an inverted sign; the reference baseline belongs to the family of uncertainty-based self-assessment probes [7]. The corpus of § 4 is built on `hendrycks MATH` [8]; task and label are defined in the proposal [9].

2 Setup and constraints

Submissions implement `are_robust(model_id, reasoning_effort, problems)`; ingestion calls it once per model group, and scoring returns exactly `accuracy`, `coverage` and `invalid_predictions`, all over the total case count. A run gets 3600 s on one 96 GB GPU, with the checkpoints cached and no internet. Test sizes are not published but follow from the score grid, since accuracy is a multiple of 1/n; constant submissions give n and the base rate:

phase	track	n	robust	best constant
v7 (until 2026-09-16)	Small	19	10	`always-true` 0.5263
v8 (from 2026-09-16)	Small	14	7	tie, 0.5000
v8	Main	21	10	`always-false` 0.5238

On 2026-09-16 the organizers replaced the working-phase problems, so everything § 6 measured about the *composition* of the v7 test describes a test that no longer exists. Our only compute was a CPU environment pinned to the container's exact runtime with a 135M stand-in model; given empty evaluation logs we consider that replication mandatory (Appendix A lists four bugs it caught and one it did not).

3 What a 14-case leaderboard can measure

Power. We model a method as a scorer with a given ROC AUC (binormal [10]: $\mu = \sqrt{2}\,\Phi^{-1}(\mathrm{AUC})$), apply the natural label-free rule — predict "robust" for the half of the batch above its median, the rule our own method used — and simulate the score 200,000 times (`report/power.py`). For the Small track (n = 14, best constant 7/14):

true AUC	E[accuracy]	sd (cases)	P(score > constant)	P(score $\geq$ constant + 3)
0.50	0.5003	1.95	0.500	0.144
0.65	0.6091	1.90	0.799	0.406
0.80	0.7294	1.75	0.967	0.771
0.90	0.8243	1.54	0.997	0.951

A coin beats or ties the constant half the time and the score's standard deviation is ± 1.9 cases. Figure 1 turns this into power against test size: AUC 0.65, the upper end of what the label-free signals of § 5 reach on the organizers' own olympiad data, is detected on the Small track 8.5% of the time by the conservative test and 17.4% by the exact one (Main: 15.6% and 23.1%; full tables in Appendix C). The rule we adopted early follows: **a public score may forbid a candidate, never select one.** A coin reaches 10/14 with probability 0.144 per submission, and the budget is 300 submissions.

The live board. On 2026-11-01, the day the working phase closed, the Small track listed 138 teams over 1,074 scored entries: 96 above the best constant and six at 14/14. The Main track listed 120 teams over 686 entries: 54 above the constant and one at 21/21. A blind coin scores 14/14 with probability $1/\binom{14}{7} \approx 3 \times 10^{-4}$, about 0.3 expected perfect entries in 1,074 — so the six perfect scores are either methods with AUC near 1 or the product of adaptive selection on the board [3], and

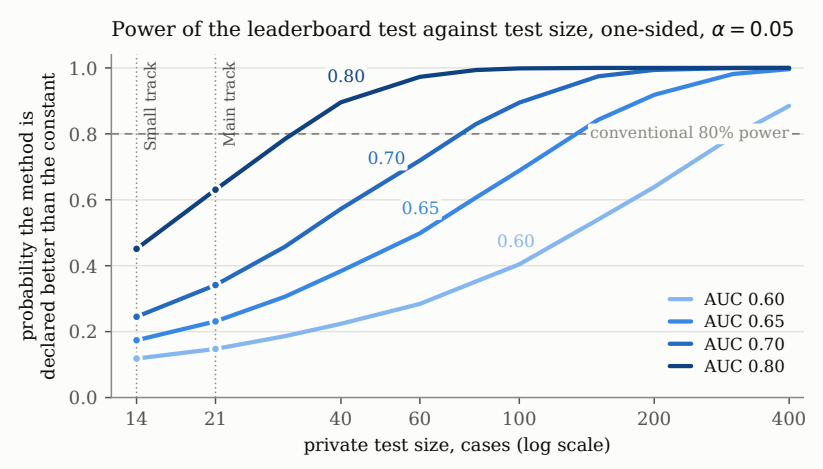


Figure 1: Probability that a one-sided test at $\alpha = 0.05$ declares a scorer of the given AUC better than the best constant, against private test size (log scale). Dots mark the two live tracks. The randomized test attains $\alpha = 0.05$ exactly; the non-randomized test is more conservative still. Both tracks sit far below the 80% line, which is crossed only between 150 and 200 cases.

14 cases cannot tell which. The organizers' decision to rank finalists on a fresh, unpublished set is the right remedy; § 3 says how large that set must be.

Where the variance lives. Across-test noise (redraw cases and samples) and within-test noise (re-run the same code on the same cases) are different questions. `report/runnoise.py` simulates the deployed method itself on fixed tests drawn from the 80 olympiad pairs of § 4 (Appendix D): at the assumed m = 4 distinct wrong answers the total sd of 1.76 cases splits into 1.59 between test draws and 0.77 within one, so **four fifths of the variance of an entry is which cases were drawn**. A three-case swing between two runs on the same test then has probability 0.03–0.07, a lower bound because the model varies only answer sampling. Our own pair sits inside that bracket: self-consistency scored 8/14 (932631) and, with one internal change (a guaranteed time reserve, Appendix A.4), 5/14 (933231) on the same test. We do not call that ordinary noise, since the code change is a live alternative; but each score alone is within one sd of the constant (exact null tails in Appendix C), so neither is evidence about the method.

4 Why the official reference solution scores below a constant

On the v7 Small track the organizers' `uncertainty-profiling` reference, a regressor over uncertainty features of a DeepSeek-R1-0528-Qwen3-8B trace, submitted unchanged, scored **0.4211 (8/19)**: below `always-true` (10/19) and `always-false` (9/19). Nineteen cases refute nothing, but the direction has an explanation checkable on published data. The labelled corpus a supervised method would train on (`math-robust-final.csv`: 675 rows, 137 problems, all for DeepSeek-R1-0528-Qwen3-8B) is built on `hendrycks MATH` [8], and **134 of 137 problems have base accuracy 1.0**: the model solves them, and the label asks whether perturbation breaks it. At AIMO-3 difficulty an 8B model mostly does not solve the base problem at all, so the regime that dominates the test is absent from the training distribution.

We verified this on the organizers' own olympiad-difficulty release (`aimo-interp-challenge-sample-full`: 10 problems × 8 models = 80 pairs; label recomputed from `absolute_accuracy_decay == 0` and validated against `model_is_robust` on `val-sample`, 24/24). Figure 2 is the whole of it.

The rule "robust ⇔ the model does not solve the base problem" alone gives accuracy 0.838 and AUC 0.784; a cluster bootstrap over the ten problems [11] puts them at [0.775, 0.900] and [0.570, 0.897] (Appendix D). The mechanism is obvious in hindsight: decay = base − perturbed, and a model at zero has nothing to lose. This one fact explains three otherwise separate numbers: the v7 robust fraction of 0.526 against 0.263 in the hendrycks corpus, the reference solution's sub-constant score,

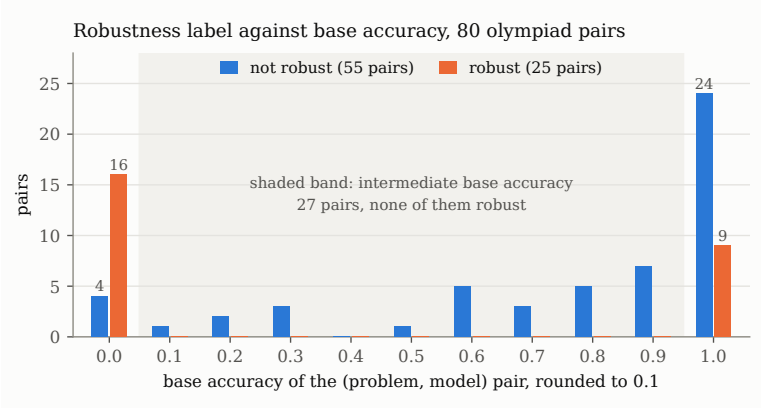


Figure 2: All 80 (problem, model) pairs, binned by base accuracy on the unperturbed problem. The shaded intermediate band holds 27 pairs and none is robust; 16 of the 25 robust pairs sit at base accuracy exactly zero, where the model had nothing to lose under perturbation.

and the failure of our own text-only feature (Appendix E). It is a property of the task at olympiad difficulty, not of any method, and the most transferable finding here.

5 Fixing the sign of a label-free proxy before submitting

A method that never sees answers needs a proxy for base accuracy. The natural one is self-consistency [6], with the intuitive sign "high agreement → stable → robust". Simulating k answers per pair from the measured base accuracies of the 80 pairs, with wrong answers spread over m distinct values:

m (distinct wrong answers)	1	2	4	8
AUC, high agreement → robust	0.745	0.439	0.366	0.330
AUC, low agreement → robust	0.255	0.561	**0.634**	**0.670**

The intuitive sign wins only when the model always repeats the same wrong answer (m = 1). We could not measure m without GPU traces, so we assumed a 4–8B model wrong on an olympiad problem is wrong in scattered ways (m $\geq$ 4) and fixed **low agreement → robust** — the one load-bearing assumption of the method, stated as such. Every signal we could evaluate on these pairs sits in AUC 0.63–0.78, exactly where § 3 shows the board cannot resolve, and the sign was fixed before any submission and never revisited: choosing a sign by a 14-bit leaderboard is the cheapest possible overfit, and after 7/21 on the Main track (933057) we did not submit the inverted rule.

The method (`solutions/self-consistency`) is deliberately minimal: 8 samples per problem at temperature 0.8, answers read only from `ANSWER:` or `\boxed{}`, agreement $\sum_a c_a(c_a - 1)/k(k-1)$, and **robust** $\Leftrightarrow$ **agreement** $\leq$ **the median of its own batch**, with no leaderboard-derived threshold and no labelled data. Scores: 8/14 and 5/14 on the Small track, 7/21 on the Main. Of our eleven pre-registered predictions (Appendix F), every arithmetic one came out exact and every one about a score failed.

6 The scoring channel is a predicate oracle

Ingestion catches exceptions **per model group**: if `are_robust` raises for one model, its cases become `invalid_predictions` while other groups stay valid, and all three numbers are fractions of the *total* case count. A submission that answers `True` where a predicate P(model_id) holds and raises elsewhere therefore reports $n \cdot \text{coverage} = |\{P\}|$ and $n \cdot \text{accuracy} = |\{P \wedge \text{robust}\}|$: one submission answers one question about the private test with two integers. Label extraction from a leaderboard is a known hazard [3, 4] that this boundary neither creates nor removes; what it adds is the **coverage** channel, which counts private cases satisfying any predicate the container can evaluate.

We used it on 2026-09-09 and 09-10: fourteen probes on the v7 Small track, each a constant over a `model_id` or `AutoConfig` predicate, counted the composition of that test by architecture

(Appendix B). We extracted no case-level labels, ran no problem-text probe, and no result entered any method; the one thing it told us scientifically was that the only labelled corpus covered 4 of 19 cases. That test was retired on 2026-09-16. We flagged the property to the organizers by email on 2026-09-19; this report is its first full description, and we leave its classification under the rules to them. Mitigations, in increasing cost: withhold the scoring numbers until the deadline; quantise or noise them [4]; invalidate the whole submission on any exception; enlarge the test.

7 Recommendations

For organizers: **report an interval beside each score**, which communicates at zero cost what § 3 argues at length; **rank finalists on a larger fresh set** — 200 cases is the order of magnitude, and even 60–100 changes the picture; **close the coverage channel** (§ 6); **return the last lines of `prediction_stdout`**, which would have saved two of the bugs in Appendix A; and **publish per-model scores** after the deadline. For participants: measure the sign of a proxy on the 80 olympiad pairs before spending a submission, do not select variants by score, and replicate the container locally.

Limitations

We never trained a white-box probe, because we had no GPU; the hidden-state baseline we prepared was never submitted, its artifact being trained on one model's representations while that model covers a minority of test cases. Our statistical claims rest on a binormal scorer and the specific median rule; other rules shift the constants but not the order of magnitude, the limit being that the score is a sum of 14 or 21 Bernoulli terms. Our deployed rule takes the median within each model group (3–4 cases), which is noisier than the batch median simulated in § 3, so the power table is if anything optimistic about the method. The within-test model of § 3 varies only answer sampling, so it bounds run-to-run variability from below, and its parameter m is assumed, not measured. The reconstruction in § 6 describes the v7 test, replaced on 2026-09-16. The live-board figures of § 3 are a snapshot of the working-phase board, not the final ranking, which the organizers compute on a fresh set. And the central negative claim — that no signal was demonstrated — says what the evaluation can show, not that the signal is absent: § 4 and § 5 both measure AUCs in the 0.63–0.78 range on the organizers' own olympiad data.

AI Use Statement

In this work, we used generative AI tools for ideation, coding, analysis and writing. The work was carried out by AI coding agents (Anthropic Claude models, run through Claude Code) under the author's direction on a CPU-only server: the agents proposed and designed the measurements, wrote all submission bundles and analysis scripts, ran them, interpreted the outputs, produced the figures, and drafted and condensed this text; a second model (OpenAI Codex) and a second Claude model were used as independent reviewers of the statistics and of the decisions to stop or continue a direction. We did not use generative AI tools to produce any reported number: every figure in this paper is the output of a seeded script in the public repository (Appendix G) or a score returned by the Codabench scorer, and no model output was used as data or as a label. We have reviewed all AI-assisted work and take responsibility for the final content of this paper.

References

[1] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky. With little power comes great responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9263–9274, 2020.

[2] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou, V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pages 747–769, 2021.

[3] A. Blum and M. Hardt. The Ladder: a reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37, pages 1006–1014, 2015.

[4] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.

[5] R. Roelofs, S. Fridovich-Keil, J. Miller, V. Shankar, M. Hardt, B. Recht, and L. Schmidt. A meta-analysis of overfitting in machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)* 32, pages 9179–9189, 2019.

[6] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.

[7] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, et al. Language models (mostly) know what they know. arXiv:2207.05221, 2022.

[8] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2103.03874.

[9] AIMO Interpretability Challenge organizers. Competition proposal. arXiv:2607.13899, 2026.

[10] J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.

[11] C. A. Field and A. H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B*, 69(3):369–390, 2007.

Appendix A. Engineering: what the empty logs hid

Evaluation returns `prediction_stdout` and `scoring_stdout` empty. Every bug below was therefore either caught locally, in a CPU environment pinned to the container's exact library versions with a 135M stand-in model, or not caught at all. We list them because each cost a submission and each will recur for anyone generating traces inside the container.

A.1 The `<think>` tag is in the prompt, not in the generation. The chat templates of `Qwen/Qwen3.5-4B`, `Skywork/Skywork-OR1-Math-7B` and `allenai/Olmo-3-7B-Think` open `<think>` at the end of the prompt; `deepseek-ai/DeepSeek-R1-0528-Qwen3-8B` leaves it to the model. A first version of our forced-answer step searched for the tag in the generated text only, and so appended `ANSWER:` *inside* an unclosed reasoning block on three models out of four. That version (932626) scored 0.3571 (5/14); the corrected one (932631), submitted before the first score was known, scored 0.5714 (8/14). § 3 applies: we do not claim the difference is measured — only that the bug was real and was found by inspecting templates, not scores.

A.2 A lossy tokenizer round-trip. With `transformers 5.13`, the DeepSeek tokenizer decodes its own ids without whitespace: `"Step 1: add 2 and 2.\nANSWER: 4"` becomes `"Step1:add2and2.ANSWER:4"`. Any forced-continuation implementation that decodes a trace to text and re-encodes it feeds the model a mangled copy of its own reasoning — on the one model for which public labels exist. We operate entirely on token ids and never round-trip.

A.3 Harmony channels. `openai/gpt-oss-120b` (Main track only) has no `<think>` tags; it writes `<|channel|>analysis<|message|> ... <|end|> ... <|start|>assistant<|channel|>final<|message|>`. Appending a bare `ANSWER:` request lands inside the analysis channel. The correct continuation for a truncated trace is `<|end|><|start|>assistant<|channel|>final<|message|>ANSWER:`.

A.4 A forcing step that disappears under time pressure. Our budget-forcing step ran under the guard `if pending and deadline - now > 20.0`. When the main generation consumes its whole slice (the `max_time` argument of `generate` fires), fewer than 20 seconds remain, the forcing pass **never runs**, every truncated sample is left without an answer, each becomes its own unique label, agreement collapses to 0 for *every* problem in the batch, and the median rule degenerates to all-`True`. We reproduced this as a controlled A/B on CPU (old branch: 0 forcing calls, answers `[None, None]`, agreement 0.0; new branch: 1 forcing call, answers `['12','15']`, a genuine disagreement) and fixed it by reserving 15% of each call's slice, floor 20 s, ceiling 90 s, for the forcing pass, with the guard removed. The fix is correct and did not help: 5/14, and the run finished in ≤ 41 minutes of a 52.5-minute budget, so on the Small track (225 s per problem) time was never the binding constraint. It plausibly was on the Main track (150 s per problem, plus a 62 GB model),

but with empty logs this remains a hypothesis. This is also the concrete mechanism that keeps the § 3 within-test model a lower bound: it is a run-level effect that changes every prediction in a batch at once and is invisible to a model that varies only answer sampling. **General lesson:** a label-free method whose decision rule is relative to a batch has a degenerate mode that scores exactly the constant — design the fallback so that the degenerate score is *distinguishable* from the method's fallback score, or you cannot tell "no signal" from "no run". We chose the all-`True` fallback on the Main track for exactly this reason, even though all-`False` scores higher there.

A.5 Operational. Queue latency ranged from 3 minutes to 5.5 hours across our submissions, so a same-day plan that depends on a result is unsafe; `model_id` values in the working phase are plain HF repository names that load directly, and any alias-rewriting map is at best useless.

Appendix B. The v7 census, probe by probe

Fourteen submissions on the v7 Small track (n = 19, 10 robust). Each answers a constant `True` where the predicate holds and raises elsewhere, so `n * coverage` is the size of the subgroup and `n * accuracy` the number of robust cases in it (§ 6).

question	submission	n·coverage	n·accuracy
name contains `deepseek`/`qwen3-8b`	920381	4	2
resolves to `Olmo-3-7B-Instruct`	920398	6	3
resolves to a large cached model (120B/27B)	920400	0	0
`AutoConfig` loads the id directly	920403	19	10
among the 9 unknowns: `model_type` ∈ {qwen3, olmo, gpt_oss, gemma}	920428	3	2
among the 9: `llama` / `mistral` / `phi` / rest	920913-5, 920917	0 each	0
among the 9: `qwen2*`	920916	6	3
among the 6 qwen2: `hidden_size >= 3584`	920921	6	3
among the 6 qwen2: `deepseek` / `qwen25` in `_name_or_path`	920919-20	0	0
among the trio: `model_type == qwen3`	920922	3	2

Final composition of the 19-case v7 Small test:

group	cases	robust
`deepseek-ai/DeepSeek-R1-0528-Qwen3-8B`	4	2
other `qwen3`	3	2
`qwen2`, 7B class	6	3
`allenai/Olmo-3-7B-Instruct`	6	3

Appendix C. Leaderboard statistics in full

Main Track, v8 (n = 21, 10 robust, best constant 11/21); the Small track version is in § 3.

true AUC	E[accuracy]	sd (cases)	P(score > constant)	P(score $\geq$ constant + 3)
0.50	0.4991	2.34	0.410	0.136
0.60	0.5701	2.31	0.669	0.327
0.65	0.6074	2.29	0.786	0.463
0.70	0.6443	2.25	0.875	0.603
0.80	0.7241	2.10	0.976	0.860
0.90	0.8167	1.80	0.999	0.986

Power against test size, both tests. The non-randomized test rejects when the score strictly exceeds the critical value; because the score is discrete it cannot attain α = 0.05 and is conservative. The randomized test additionally rejects with probability γ at the critical value, attaining α = 0.05 exactly; it is the curve plotted in Figure 1.

n (balanced)	reject if score >	actual α (non-rand.)	power at AUC 0.65 (non-rand.)	power at AUC 0.65 (randomized)
14	10/14	0.014	0.085	0.174
21	14/21	0.022	0.156	0.231
40	26/40	0.013	0.192	0.384
60	36/60	0.035	0.449	0.498
100	58/100	0.036	0.642	0.688
150	86/150	0.025	0.757	0.843
200	112/200	0.033	0.891	0.918
300	164/300	0.042	0.979	0.982
400	216/400	0.047	0.996	0.996

The non-randomized column is not monotone in n (0.192 at n = 40 against 0.449 at n = 60) because the attainable α jumps with the discreteness of the score; this is an artefact of the test, not of the benchmark, and is the reason Figure 1 plots the randomized version.

Exact null tails for our three method submissions. A submission predicting `True` on s of n cases, with n_pos robust cases and no signal, gets t ~ Hypergeom(n, n_pos, s) of them right among its `True` answers, so the number correct is n $-$ n_pos $-$ s + 2t.

submission	track	score	E[correct] under null	tail
932631 self-consistency	Small	8/14	7.00	P($\geq$ 8) = 0.500
933231 same + time reserve	Small	5/14	7.00	P($\leq$ 5) = 0.296
933057 same, Main track	Main	7/21	10.5	P($\leq$ 7) = 0.063–0.135 (s unknown)

Appendix D. The within-test noise model, and clustered intervals

The model (`report/runnoise.py`, used for § 3). Population: the 80 (problem, model) pairs of `aimo-interp-challenge-sample-full`, with their measured base accuracies and re-computed labels. A fixed test is 7 robust + 7 not robust drawn without replacement, matching the measured composition of the v8 Small track; model groups (4/4/3/3) are assigned independently of the labels. A run draws k = 8 answers per case — the correct one with probability b, otherwise one of m distinct wrong answers uniformly — computes agreement $(\sum_v c_v^2 - k)/(k(k-1))$, and applies `robust <=> agreement <= median + 1e-9`, copied verbatim from the submitted decision rule. 3,000 fixed tests $\times$ 120 runs; `sd within test` is the root mean square of the

per-test run variance and `sd between tests` the spread of per-test means, so their squares sum to the total variance of a score. Full output, including m = 1 (degenerate: every score is exactly the constant, because with a single wrong answer value agreement saturates and the median rule ties everything) and both decision rules, is in `report/runnoise-output.txt`.

m	rule	E[score]	sd within test	sd between tests	E|Δ|	P(|Δ| ≥ 3)
2	batch median	7.96	0.90	1.38	0.83	0.067
4	batch median	8.42	0.77	1.59	0.62	0.043
8	batch median	8.67	0.67	1.68	0.51	0.032
4	per-model median	7.88	0.81	1.49	0.72	0.037

The per-model median we deployed costs about half a case of expected score against the batch median. The across-test figure of § 3, P(|Δ| ≥ 3) = 0.265 between two independent scores (sd 1.95 cases), additionally redraws the cases and is the upper bound for the same question.

The model is a lower bound on run-to-run variability by construction: it varies only answer sampling, holding fixed the trace-level behaviour that Appendix A.4 shows can change an entire batch at once, and it assumes wrong answers are exchangeable across the m values. The upper bound is the across-test figure of § 3, which additionally redraws the cases. Both are reported rather than one being presented as the answer.

Knife-edge tests. The run-to-run tail is not uniform across test draws: at m = 4 the 95th percentile of P(|Δ| ≥ 3) over fixed tests is 0.23, so an unlucky case draw puts an entrant where re-running the identical method reshuffles their rank.

The perturbation alternative of § 5. Decays of different perturbation types correlate at Spearman 0.6–0.8, and the rule "zero decay under one cheap perturbation type → robust" reaches AUC 0.65 on 70 pairs (`typos` 0.650, `distract` 0.650, `rephrase` 0.620, `rename` 0.550) — the same ceiling as plain agreement, for twice the generation cost.

Clustered intervals for § 4 (`report/ci80.py`, cluster bootstrap over the 10 problems, 5,000 resamples):

quantity	point	95% CI (clustered by problem)
accuracy of "base == 0 → robust"	0.838	[0.775, 0.900]
AUC of the same rule	0.784	[0.570, 0.897]
AUC of extremeness max(base, 1−base)	0.745	[0.653, 0.830]
AUC of −base_accuracy	0.695	[0.402, 0.865]

Appendix E. A text-only feature that works on its corpus and not on the task

Before the mechanism of § 4 was clear we built a text-prior model on the 137 hendrycks problems: 34 hand features, grouped 5-fold CV × 20 repeats, OOF ROC AUC 0.707 (random forest). A single feature, `log1p(n_words)`, gives 0.737 with bootstrap CI [0.648, 0.821], and a label-permutation test over 150 shuffles of the same pipeline gives null AUC 0.508 ± 0.077 against 0.714, p = 0.0066. The signal is real on its corpus, the direction is "longer statement → more robust", and it transfers across hendrycks splits (0.827 / 0.725 train↔test, against 0.583 / 0.590 for the 34-feature forest).

It does not transfer to the task. On the 80 olympiad pairs `n_words` has AUC 0.555, and the fitted decision boundary (17.4 words) lies below the *minimum* statement length in the validation sample (22 words), so the rule degenerates into `always-true`. This was pre-registered as prediction 8

(Appendix F) and the submission confirmed it exactly: 0.5263, identical to the constant, on all 19 v7 cases. We record it as a clean negative transfer result with a diagnosed cause, not as a failure of the feature: olympiad statements are uniformly long, so a length prior calibrated on a corpus with short statements has no dynamic range left.

Appendix F. The eleven pre-registered predictions

Written into the project log before the corresponding submission and not edited afterwards: five clearly wrong, one half-right, five right.

#	prediction (written before the run)	outcome	verdict
1	test is mixed; labelled model covers 40–60% of cases, accuracy 0.10–0.25	coverage 21%, accuracy 0.105	direction right, magnitude wrong 2×
2	of the 9 unknown cases, all 9 are `qwen3`/`olmo`/`gpt_oss`/`gemma`	3 of 9	**wrong (3×)**
3	the remaining 6 split as llama 2 / mistral 1 / phi 1 / qwen2 1 / other 1	all 6 are `qwen2`	**wrong (completely)**
4	the five architecture probes must sum to coverage 6/19 and accuracy 3/19	exact on both	right
5	the 6 `qwen2` cases are 7B-class (`hidden_size` $\geq$ 3584)	6 of 6	right
6	the 6 carry `deepseek` or `qwen25` in `_name_or_path`	0 of 6	**wrong**
7	the remaining trio is `model_type == qwen3`	3 of 3	right
8	`text-prior` degenerates to the constant on the private test (0.5263)	0.5263	right (a tautology, as we said at the time)
9	self-consistency on the Small track $\approx$ 8/14, plausible 6–10	8/14	right — and, per § 3, uninformative
10	self-consistency transfers to the Main track: $\geq$ 12/21, plausible 10–14	7/21	**wrong**
11	restoring forced decoding lifts the Small track score to $\geq$ 9/14	5/14	**wrong**

The arithmetic predictions — consistency checks on the census, `AutoConfig` fields, the tautological degeneration of the text-prior rule to a constant — came out exact, and every prediction about a leaderboard score or about the test's diversity failed. The exact predictions are deductions from previous measurements; the failures are guesses about something we had not measured. We report

the record in full, including the hits, because a report that lists only its confirmed hypotheses — in a setting with this much sampling noise — should not be believed. Two of the predictions were *thresholds* fixed in advance, and both closed a direction when they were missed rather than licensing another variant: after 5/14 we stopped submitting variants of the method on both tracks.

Appendix G. Reproducibility, every submission, and where each number comes from

Repository. `https://github.com/oleksandr-barskyi/aimo-interp-report` holds `solutions/` (every submitted bundle as a source directory), `analysis/` and `report/` (all scripts, their saved outputs, both figures and this paper's source). No labelled data beyond the organizers' public releases was used: `math-robust-final.csv` from `aimo-interp/baselines`, `aimo-interp/val-sample`, and `aimo-interp-challenge-sample-full`. No model was trained for any submitted method; the only fitted object, the four-number length model of `text-prior`, is in the bundle (`length_logreg.json`) and is produced by `analysis/final_model.py`.

Rebuilding a bundle. Each bundle is the deterministic ZIP of one directory of `solutions/`, made by `scripts/build.py` of `aimo-interp/getting-started` at commit `de79405`; Small-track bundles carry the organizers' `small.txt` marker. The self-consistency bundle has no artifact and no dependency beyond the evaluation image, so the scores below are reproducible from the repository alone, up to the sampling noise quantified in § 3.

Which script gives which number. `report/power.py` produces every power number in § 3 (about a minute on a CPU, fixed seed) and the Appendix C tables; `report/runnoise.py` the within-test model of § 3 and Appendix D; `report/ci80.py` the contingency table and clustered intervals of § 4; `analysis/aimo_reference.py` the 80-pair mechanism of § 4 and § 5; `report/figures.py` both figures, with `fig1-power.csv` as the underlying grid; `report/submissions.py` the table below and the live-board figures of § 3, saved with their snapshot time in `report/submissions-output.txt`. `analysis/` also holds the corpus work behind Appendix E (`cv.py`, `nulltest.py`, `nested.py`, `confound.py`, `ci.py`). Local replication used `torch 2.12.1+cpu`, `transformers 5.13.0`, `tokenizers 0.22.2`, `accelerate 1.13.0`, `safetensors 0.8.0`, and the harness of `aimo-interp/getting-started` at `de79405`.

Standing. The working-phase board was reset on 2026-10-15 and lists only entries made after it; we made no submission after 2026-09-19, so we have no entry on it. Measured against the board snapshot of 2026-11-01 07:04 UTC, our best Small-track score (8/14) would rank about 70th of 139 teams and the Main-track constant (11/21) about 55th of 121. The final ranking is computed by the organizers on a fresh, unpublished set and was not known when this report was written. The submissions named for prize evaluation are 932631 (Small track) and 933057 (Main track), both `self-consistency`.

All submissions (Codabench user `obarskyi`, competition 16180; dates are 2026, UTC; bundle names without the `solution-` prefix and `-small` suffix). v7 is the 19-case Small test retired on 2026-09-16; v8 is the 14-case Small and 21-case Main test that replaced it. Scores are fractions of the full test; 920251 is a child submission created on the Main track by mistake (Small marker, so every case invalid), and two entries (canc.) were cancelled in the queue before scoring.

id	date	test	bundle	acc.	cov.	inv.
920251	09-09 14:40	v7 Main	always-true	0	0	21
920252	09-09 14:40	v7 Small	always-true	0.5263	1	0
920255	09-09 14:42	v7 Small	uncertainty-profiling	0.4211	1	0
920351	09-09 16:01	v7 Small	text-prior	0.5263	1	0
920372	09-09 16:44	v7 Small	trace-length	0.4737	1	0
920381	09-09 16:50	v7 Small	census	0.1053	0.2105	15
920398	09-09 17:12	v7 Small	census-olmo	0.1579	0.3158	13
920400	09-09 17:15	v7 Small	census-rest	0	0	19
920403	09-09 17:17	v7 Small	census-loadable	0.5263	1	0
920428	09-09 17:33	v7 Small	census-family	0.1053	0.1579	16

id	date	test	bundle	acc.	cov.	inv.
920913	09-10 06:03	v7 Small	census-llama	0	0	19
920914	09-10 06:03	v7 Small	census-mistral	0	0	19
920915	09-10 06:03	v7 Small	census-phi	0	0	19
920916	09-10 06:03	v7 Small	census-qwen2	0.1579	0.3158	13
920917	09-10 06:03	v7 Small	census-other	0	0	19
920919	09-10 06:07	v7 Small	census-q2name	0	0	19
920920	09-10 06:07	v7 Small	census-q2qwen25	0	0	19
920921	09-10 06:07	v7 Small	census-q2large	0.1579	0.3158	13
920922	09-10 06:07	v7 Small	census-trio-qwen3	0.1053	0.1579	16
928280	09-16 20:11	v8 Main	always-true	canc.	canc.	canc.
929069	09-16 22:35	v8 Small	always-true	canc.	canc.	canc.
932602	09-18 22:14	v8 Small	always-true	0.5000	1	0
932603	09-18 22:14	v8 Main	always-true	0.4762	1	0
932604	09-18 22:15	v8 Main	always-false	0.5238	1	0
932626	09-18 23:02	v8 Small	self-consistency	0.3571	1	0
932631	09-18 23:10	v8 Small	self-consistency	0.5714	1	0
933057	09-19 07:44	v8 Main	self-consistency	0.3333	1	0
933231	09-19 09:42	v8 Small	self-consistency	0.3571	1	0